

End-to-End Efficiency in Keyword Spotting: A System-Level Approach for Embedded Microcontrollers

Pietro Bartoli*, Tommaso Bondini*, Christian Veronesi*, Andrea Giudici*, Niccolò Antonello[†], Franco Zappa*

*Dept. of Electronics, Information and Bioengineering, Politecnico di Milano, Milan, Italy

[†]Smart Eyewear Laboratory, EssilorLuxottica, Milan, Italy

Abstract—Keyword spotting (KWS) is a key enabling technology for hands-free interaction in embedded and IoT devices, where stringent memory and energy constraints challenge the deployment of AI-enabled devices. In this work, we systematically evaluate and compare several state-of-the-art lightweight neural network architectures, including DS-CNN, LiCoNet, and TENet, alongside our proposed Typman-KWS (TKWS) architecture built upon MobileNet, specifically designed for efficient KWS on microcontroller units (MCUs). Unlike prior studies focused solely on model inference, our analysis encompasses the entire processing pipeline, from Mel-Frequency Cepstral Coefficient (MFCC) feature extraction to neural inference, and is benchmarked across three STM32 platforms (N6, H7, and U5). Our results show that TKWS with three residual blocks achieves up to 92.4% F1-score with only 14.4k parameters, reducing memory footprint without compromising the accuracy. Moreover, the N6 MCU with integrated neural acceleration achieves the best energy-delay product (EDP), enabling efficient, low-latency operation even with high-resolution features. Our findings highlight the model accuracy alone does not determine real-world effectiveness; rather, optimal keyword spotting deployments require careful consideration of feature extraction parameters and hardware-specific optimization.

Index Terms—Keyword Spotting, Edge AI, MFCC, MCU, EDP

I. INTRODUCTION

Keyword spotting (KWS) refers to the detection of predefined words in a continuous audio stream, enabling hands-free interaction with devices and forming a fundamental component of voice-controlled systems and modern IoT applications [1]. Widely deployed consumer platforms like *Amazon Alexa*, *Google Assistant*, and *Apple Siri* exploit KWS to recognize specific wake words (“Alexa,” “Hey Google,” and “Hey Siri”) allowing for intuitive, hands-free voice interfaces [2]. In these systems, the KWS module operates locally to reduce latency and maintain privacy, activating cloud-based processing after the wake word is detected for advanced tasks like speech recognition [3], [4]. However, while these KWS algorithms run efficiently on consumer devices such as smart speakers and smartphones, with their greater processing power, deploying them on battery-powered microcontrollers (MCUs) introduces significant challenges [5]. MCUs operate under strict memory

and computation constraints, requiring KWS solutions to carefully balance responsiveness and accuracy [6], [7], while also minimizing energy consumption, a critical factor in battery-powered device. This trade-off between latency, accuracy, and efficiency becomes even more complex as recent deep learning approaches, although improving accuracy [8], often exceed the computational and memory budgets of embedded hardware [9].

Moreover, the audio pre-processing chain imposes additional overhead, particularly through Mel-Frequency Cepstral Coefficients (MFCCs), one of the most widely adopted feature extraction methods in automatic speech processing [10]. MFCCs are acoustic features that mimic human auditory perception by reflecting the ear’s nonlinear frequency sensitivity, achieved first converting audio to a mel-scaled spectrogram and then applying dimensionality reduction via the Discrete Cosine Transform (DCT) [11].

While most previous works have focused on optimizing KWS model architectures [12]–[14], there is still a lack of comprehensive studies that evaluate both the energy and memory costs of audio pre-processing and inference on different embedded platforms. To fill this gap, in this work we systematically assess the full KWS pipeline, including audio pre-processing, feature extraction and neural network inference on three different MCUs: U5, H7, and N6.

We also introduce *Typman-KWS* (TKWS), a lightweight neural model designed for low memory and energy usage, and we compared it with different model architectures for KWS on MCUs. Through experimental analysis, we systematically quantify the trade-offs between latency, accuracy, and resource utilization, providing practical guidelines for the deployment of efficient KWS solutions in energy-constrained embedded systems.

II. EXPERIMENTAL SETUP

The experimental setup was carefully designed to achieve two principal aims, focusing on both model validation and performance profiling. First, we evaluate the performance of the proposed TKWS architecture in terms of accuracy and number of parameters, comparing it to reference models from the literature. Second, we aim to quantify the energy requirements of deploying a KWS system on MCUs by systematically analyzing how variations in MFCC feature

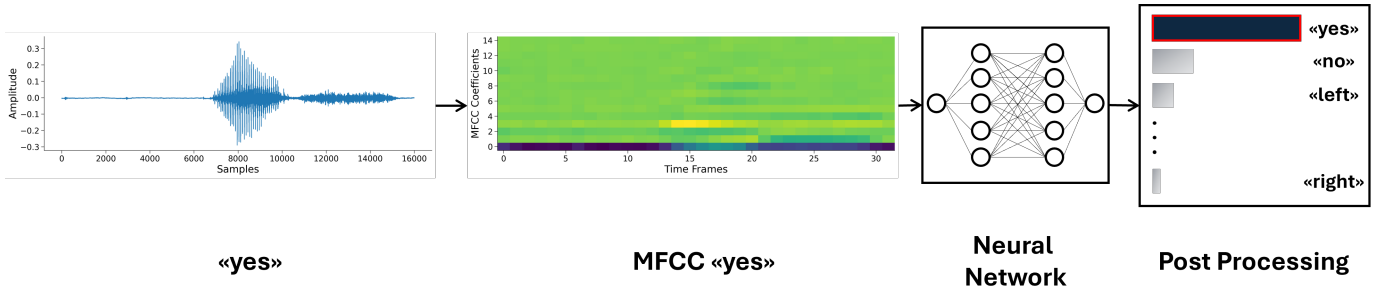


Fig. 1. Overview of the processing pipeline. The raw audio waveform is transformed into time-frequency features (*MFCC*), which are subsequently processed by a neural network. A post-processing stage is then applied to extract the most likely prediction.

extraction parameters impact the overall resource consumption of the pipeline.

A. Dataset

All models were trained and evaluated on the *Google Speech Commands Dataset (GSCD)* [15] v0.02, using a subset of 10 keyword classes —*yes*, *no*, *up*, *down*, *left*, *right*, *on*, *off*, *stop*, and *go*— for the classification task.

To improve robustness to real-world conditions, training samples were augmented with noise from the GSCD, including both synthetic and real background sounds, mixed at a signal-to-noise ratio (SNR) sampled from $\mathcal{N}(10 \text{ dB}, 5 \text{ dB})$. The pipeline (Fig. 1) extracts and normalizes MFCC features from raw audio before neural classification. To assess the impact of feature resolution, we evaluated multiple MFCC configurations by varying the number of windows (32, 63) and Mel filter banks (15, 30). These values were selected to balance efficiency and expressiveness: 32 and 63 time frames align with SFFT sizes of 1024 and 512, avoiding zero-padding and enabling efficient computation, while 15 and 30 Mel filters represent lightweight and more detailed spectral resolutions.

B. Proposed Models

We evaluate four lightweight neural network architectures for KWS on low-power embedded platforms, including three reference models from the literature and TKWS, a custom design developed to meet the memory and energy constraints of MCUs. The evaluated architectures are:

1) *DS-CNN*: Implements depthwise separable convolutions by factorizing standard convolutions into depthwise and pointwise operations, as in the Hello Edge framework [12], to achieve a favorable balance between accuracy and resource efficiency.

2) *LiCO-Net*: Uses the small configuration LiCO-Net [16] (LicoNet-S), adapted from its original streaming inference design with causal convolutions to perform one-shot inference by modifying the padding strategy, enabling full sample processing without maintaining internal state.

3) *TENet*: Employs a multi-branch temporal convolutional architecture available in standard (TENet6) and narrow (TENet6-N) versions [17], utilizing kernel fusion at inference to efficiently aggregate multi-scale temporal features while minimizing runtime complexity.

4) *TKWS*: Inspired by the inverted bottleneck design of MobileNetV2 [18], the model is built upon stacked residual blocks that combine pointwise expansion, double 1D depthwise convolutions, and projection layers. It operates directly on MFCC temporal sequences and is implemented in two configurations, featuring either 2 (TKWS-2) or 3 (TKWS-3) residual blocks.

For all models, the classification head consists of a global average pooling layer followed by a fully connected layer with softmax activation to output class probabilities.

C. Benchmarking Procedure

The experimental evaluation was performed on three STM32 MCUs-N6 (Cortex-M55), H7 (Cortex-M7), and U5 (Cortex-M33)-representing different computational classes. All CPUs were configured to operate at their maximum frequencies: 800MHz for N6, 480MHz for H7, and 160MHz for U5. All models were quantized to 8-bit integers, and MFCC extraction leveraged each platform’s digital signal processor (DSP) for optimal performance. The evaluated models were assessed according to the following metrics:

- **Number of Parameters**: quantifies the static memory occupation on MCUs, as model weights, which must be stored in flash memory.
- **Weighted F1-score**: used as the primary classification performance metric to account for potential class imbalance in the test set.
- **Energy-Delay Product (EDP)**: a combined metric balancing latency and energy consumption, defined as the product of latency and energy consumption. This figure of merit provides a comprehensive evaluation of system-level efficiency [19], [20].

The overall measurement procedure follows the methodology described in [21]. Latency and energy were measured for the pre-processing and inference phases, and the EDP was calculated by combining these two stages, excluding the negligible post-processing step.

III. RESULTS AND DISCUSSION

In this section, we present a comprehensive evaluation, beginning with model performance (accuracy and parameter

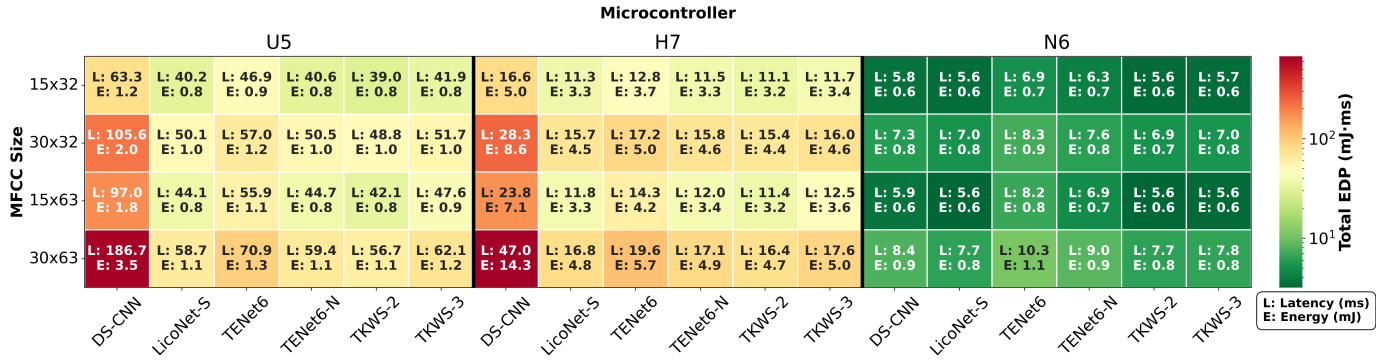


Fig. 2. Heatmap of total Energy-Delay Product (EDP) [$mJ \cdot ms$] for the end to end keyword spotting pipeline, evaluated across different MFCC sizes (Mel filter banks $\times$ #windows), model architectures, and microcontroller platforms (U5, H7, N6). Cell colors reflect EDP values (lower is better), while each cell also reports latency (L) in ms and energy (E) in mJ .

TABLE I

MODEL PERFORMANCE METRICS FOR THE KEYWORD SPOTTING TASK. FOR EACH MODEL AND MFCC SIZE (MEL FILTER BANKS $\times$ TIMESTEPS), THE TABLE REPORTS THE TOTAL NUMBER OF PARAMETERS (P) AND THE WEIGHTED F1 SCORE [%] ON THE TEST SET.

Model	15 $\times$ 32		30 $\times$ 32		15 $\times$ 63		30 $\times$ 63	
	P	F1	P	F1	P	F1	P	F1
DS-CNN	46.5k	86.0	46.5k	86.7	46.5k	91.2	46.5k	91.0
LicoNet-S	17.4k	90.1	18.3k	90.3	17.4k	93.6	18.3k	92.5
TENet6	54.2k	90.5	55.7k	91.2	54.2k	92.9	55.7k	93.2
TENet6-N	17.1k	90.5	17.9k	90.3	17.1k	91.8	17.9k	91.7
TKWS-2	4.6k	86.5	5.0k	87.4	4.6k	88.8	5.1k	87.3
TKWS-3	14.4k	90.7	14.9k	91.1	14.4k	92.4	14.9k	91.2

count) and subsequently assessing system-level efficiency on the three aforementioned MCUs.

Table I presents the classification performance and parameter count of the evaluated models. The number of parameter directly impact static memory usage, making model size a critical constraint due to the limited memory resources available on MCUs. Notably, configurations with 63 time windows consistently outperform those with 32, showing that higher temporal resolution significantly improves accuracy and must be accounted for in resource-constrained system design. Among the evaluated models, TKWS-3 achieves a high F1 score of 92.4% with only 14.4k parameters (MFCC size: 15×63). This performance closely matches that of LicoNet-S (93.6% F1, 17.4k parameters), while requiring 17% fewer parameters. Compared to DS-CNN (91.2% F1, 46.5k parameters) and TENet6 (92.9% F1, 54.2k parameters), TKWS-3 provides equal or better accuracy while reducing the memory footprint by over threefold. Even the most compact configuration of TKWS-2 (88.8% F1 with only 4.6k parameters) demonstrates competitive performance, highlighting the efficiency of the TKWS architecture in achieving high accuracy with minimal memory requirements.

Figure 2 shows the EDP for the complete KWS pipeline across different MFCC input configurations and model architectures. Among the platforms evaluated, N6 demonstrates the highest efficiency, leveraging its advanced DSP capabilities for

fast MFCC extraction and an integrated NPU that supports low-power, high-speed inference. This combination allows the platform to effectively accommodate higher temporal resolutions, such as MFCC inputs with 63 timesteps, without incurring a significant energy cost. By comparison, both H7 and U5 exhibit worse (i.e., higher) EDP values than N6 and are similar to each other, though for different reasons: the former achieves lower latency with greater energy consumption, while the latter favors energy efficiency over speed. The similarity in their EDP values indicates that the optimal platform choice should depend on the target application: H7 is preferable when quick response is critical (e.g., real-time command recognition), whereas U5 is more suitable for ultra-low-power scenarios where latency is less important.

Moreover, the analysis of the EDP highlights the interplay between model architecture and hardware capabilities. On MCUs without NPUs, 1D convolutional models consistently achieve better energy efficiency than 2D ones. For instance, DS-CNN, based on 2D convolutions, shows the highest EDP on both H7 and U5 due to its greater computational complexity and poor hardware compatibility. However, on the N6 platform, this trend shifts, as TENet models show higher EDP due to kernel sizes that exceed the NPU's optimal execution range. As a result, these operations are offloaded to the CPU, leading to increased overhead from data transfers and less efficient layer execution. These findings underscore the importance of co-designing neural architectures and hardware platforms, as the optimal balance between accuracy and efficiency is ultimately determined by their combined characteristics.

IV. CONCLUSION

This study systematically evaluates KWS models on low-power MCUs, assessing both feature extraction and neural inference under real-world conditions. Results demonstrate that efficient architectures like TKWS-2 and TKWS-3 can significantly reduce memory requirements with minimal accuracy loss, making them well-suited for always-on applications. The analysis further reveals that hardware capabilities play a critical role: platforms like N6, with integrated DSP and

NPU, achieve higher efficiency and speed, especially with complex input features, while general-purpose MCUs involve trade-offs between energy and latency. Overall, the findings highlight the need for a holistic co-design strategy that aligns model architecture with hardware characteristics to optimize performance and resource utilization in embedded AI systems.

REFERENCES

- [1] I. Lopez-Espejo, Z.-H. Tan, J. H. L. Hansen, and J. Jensen, "Deep spoken keyword spotting: An overview," *IEEE Access*, vol. 10, pp. 4169–4199, 2022.
- [2] A. H. Michaely, X. Zhang, G. Simko, C. Parada, and P. Aleksic, "Keyword spotting for google assistant using contextual speech recognition," in *2017 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2017, pp. 272–278.
- [3] P.-J. Ku, I.-F. Chen, C.-H. H. Yang, A. Raju, P. Dheram, P. Ghahremani, B. King, J. Liu, R. Ren, and P. S. Nidadavolu, "Hot-fixing wake word recognition for end-to-end asr via neural model reprogramming," in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 10 816–10 820.
- [4] M. Jabbeen, V. Kumar, and R. Sen, "SpotOn: Adversarially robust keyword spotting on resource-constrained IoT platforms," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*. New York, NY, USA: ACM, Jul. 2024.
- [5] M. G. Ulkar and O. E. Okman, "Ultra-low power keyword spotting at the edge," 2021. [Online]. Available: <https://arxiv.org/abs/2111.04988>
- [6] C. Cioflan, L. Cavigelli, M. Rusci, M. de Prado, and L. Benini, "On-device domain learning for keyword spotting on low-power extreme edge embedded systems," 2024. [Online]. Available: <https://arxiv.org/abs/2403.10549>
- [7] J. Wang and S. Li, "Keyword spotting system and evaluation of pruning and quantization methods on low-power edge microcontrollers," 2022. [Online]. Available: <https://arxiv.org/abs/2208.02765>
- [8] S. O. Arik, M. Kliegl, R. Child, J. Hestness, A. Gibiansky, C. Fougner, R. Prenger, and A. Coates, "Convolutional recurrent neural networks for small-footprint keyword spotting," 2017. [Online]. Available: <https://arxiv.org/abs/1703.05390>
- [9] L. S. Martinez-Rau, Q. N. P. Vu, Y. Zhang, B. Oelmann, and S. Bader, "Adaptive noise resilient keyword spotting using one-shot learning," 2025. [Online]. Available: <https://arxiv.org/abs/2505.09304>
- [10] J. Vreča, R. Pilipović, and A. Biasizzo, "Hardware–software co-design of an audio feature extraction pipeline for machine learning applications," *Electronics*, vol. 13, no. 5, p. 875, Feb. 2024. [Online]. Available: <http://dx.doi.org/10.3390/electronics13050875>
- [11] Z. K. Abdul and A. K. Al-Talabani, "Mel frequency cepstral coefficient and its applications: A review," *IEEE Access*, vol. 10, pp. 122 136–122 158, 2022.
- [12] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello edge: Keyword spotting on microcontrollers," 2018. [Online]. Available: <https://arxiv.org/abs/1711.07128>
- [13] P. Blouw, G. Malik, B. Morcos, A. R. Voelker, and C. Eliasmith, "Hardware aware training for efficient keyword spotting on general purpose and specialized hardware," in *tinyML Conference*, 2021.
- [14] S. Mittermaier, L. Kürzinger, B. Waschneck, and G. Rigoll, "Small-footprint keyword spotting on raw audio data with sinc-convolutions," *arXiv preprint arXiv:1911.02086*, 2019.
- [15] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018. [Online]. Available: <https://arxiv.org/abs/1804.03209>
- [16] H. Yang, Z. Yang, L. Wan, B. Zhang, Y. Shi, Y. Huang, I. Enchev, L. Tang, R. Alvarez, M. Sun, X. Lei, R. Krishnamoorthi, and V. Chandra, "Lico-net: Linearized convolution network for hardware-efficient keyword spotting," 2022. [Online]. Available: <https://arxiv.org/abs/2211.04635>
- [17] X. Li, X. Wei, and X. Qin, "Small-footprint keyword spotting with multi-scale temporal convolution," 2020. [Online]. Available: <https://arxiv.org/abs/2010.09960>
- [18] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," 2019. [Online]. Available: <https://arxiv.org/abs/1801.04381>
- [19] V. Fratin, D. Oliveira, P. Navaux, L. Carro, and P. Rech, "Energy-delay-fit product to compare processors and algorithm implementations," *Microelectronics Reliability*, vol. 84, pp. 112–120, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S002627141830132X>
- [20] G. Datta, S. Kundu, Z. Yin, R. T. Lakkireddy, J. Mathai, A. P. Jacob, P. A. Beerel, and A. R. Jaiswal, "A processing-in-pixel-in-memory paradigm for resource-constrained tinyml applications," *Scientific Reports*, vol. 12, no. 1, p. 14396, 2022. [Online]. Available: <https://doi.org/10.1038/s41598-022-17934-1>
- [21] P. Bartoli, C. Veronesi, A. Giudici, D. Siorpaes, D. Trojaniello, and F. Zappa, "Benchmarking energy and latency in tinyml: A novel method for resource-constrained ai," 2025. [Online]. Available: <https://arxiv.org/abs/2505.15622>